NOVA IMS INFORMATION MANAGEMENT SCHOOL

Logic and strings

Computation I · Lecture 03

Karina Brotto Rebuli
krebuli@novaims.unl.pt

And the last lecture's debt

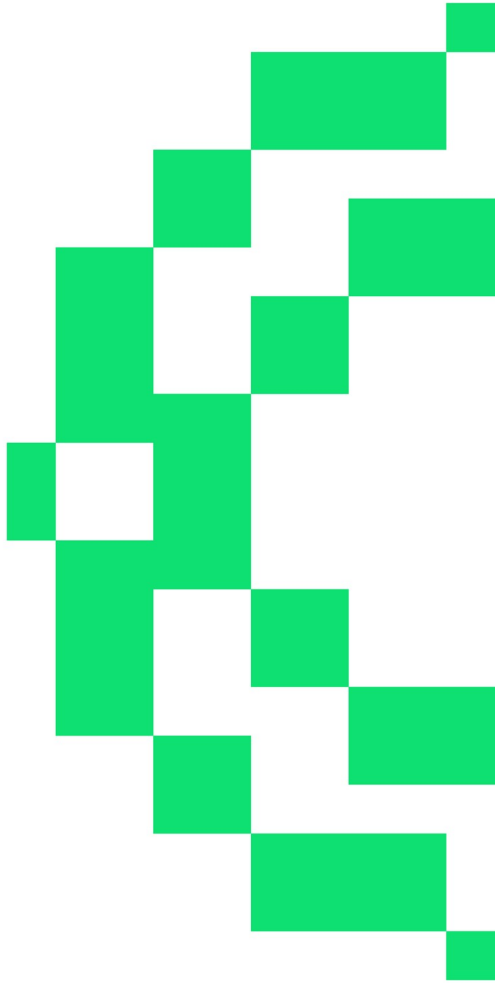
Exercise 10, replayed

```
print("10" < "9")
```

```
>>> True
```

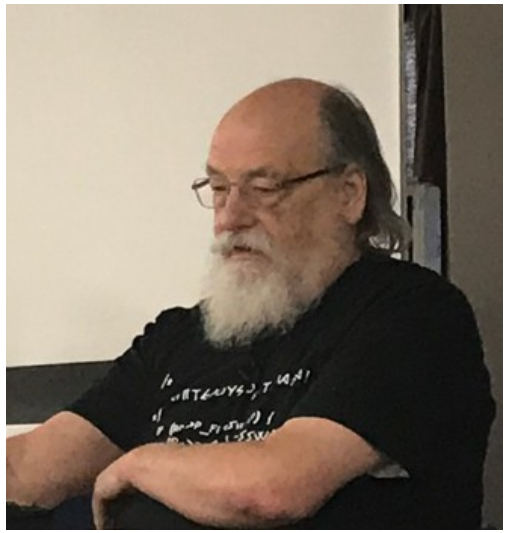
It was not what you meant.

No crash. No warning. A legal answer, but wrong for you.



Ken Thompson

1992



Wikimedia Commons

A letter is not stored as a letter. It is stored as a number. The table that makes the correspondence of a letter to a number used to have 128 places: enough for English, but not for special characters like ç, and not for Greek or Japanese. The problem: make room for every alphabet on Earth without breaking a single text already written.

Ken Thompson and a colleague worked this out at a diner, sketching it on a placemat. Every string you take apart today is stored the way that sketch decided.

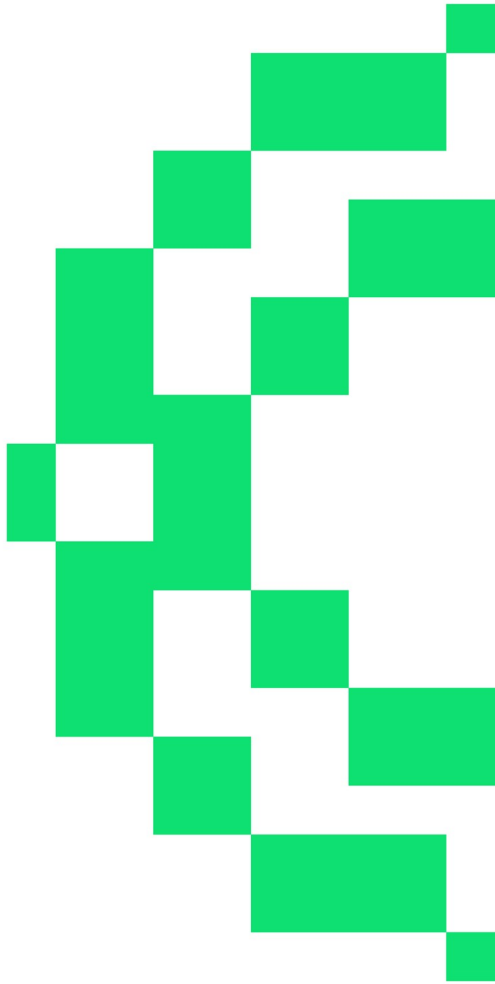
Three questions today

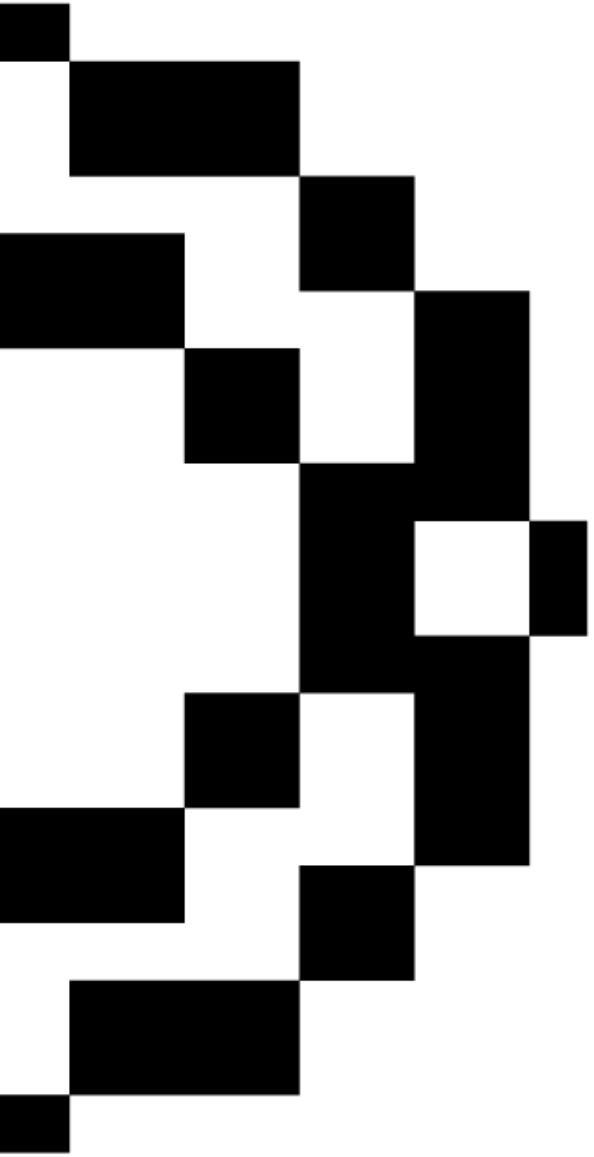
How questions combine.

What lives inside a string.

Whether a string can be modified.

```
"""and multiplies, or adds, not inverts."""
```





01.

To be *and* not to be, that's also a question!

Combining True and False values

Six ways to ask — comparison operators

Each one asks a question about two values; Python answers bool values, either True or False

`==`	is this equal to that?
`!=`	is this different from that?
`<`	is this less than that?
`>`	is this greater than that?
`<=`	is this less than or equal to that?
`>=`	is this greater than or equal to that?

Three words you already use

Everyday decisions already combine questions — before any Python

`and`

both conditions are required

You board the plane if you have a ticket `and` a passport.

`a ticket`	`True`
`a passport`	`False`
`board?`	**`False`**

`both, or nothing`

`or`

only one condition is required

You pay less at the museum if you are a student `or` a senior.

`a student`	`False`
`a senior`	`True`
`discount?`	**`True`**

`one True is enough`

`not`

inverts the condition

The shop is open when it is `not` Sunday.

`Sunday`	`True`
`open?`	**`False`**

`not flips it`

The truth tables

Python borrows the words as they are — and, or, not — and always answers with a bool

`and`

both conditions are required

A	B	A `and` B
F	F	F
F	T	F
T	F	F
T	T	T

`or`

only one condition is required

A	B	A `or` B
F	F	F
F	T	T
T	F	T
T	T	T

`not`

inverts the condition

A	`not` A
F	T
T	F

`and` multiplies, `or` adds, `not` inverts

True is 1, False is 0 — the tables are just arithmetic

`and`

both conditions are required

A	B	A `and` B
0	0	0
0	1	0
1	0	0
1	1	1

`works like A * B`

`or`

only one condition is required

A	B	A `or` B
0	0	0
0	1	1
1	0	1
1	1	1

`works like A + B`

`not`

inverts the condition

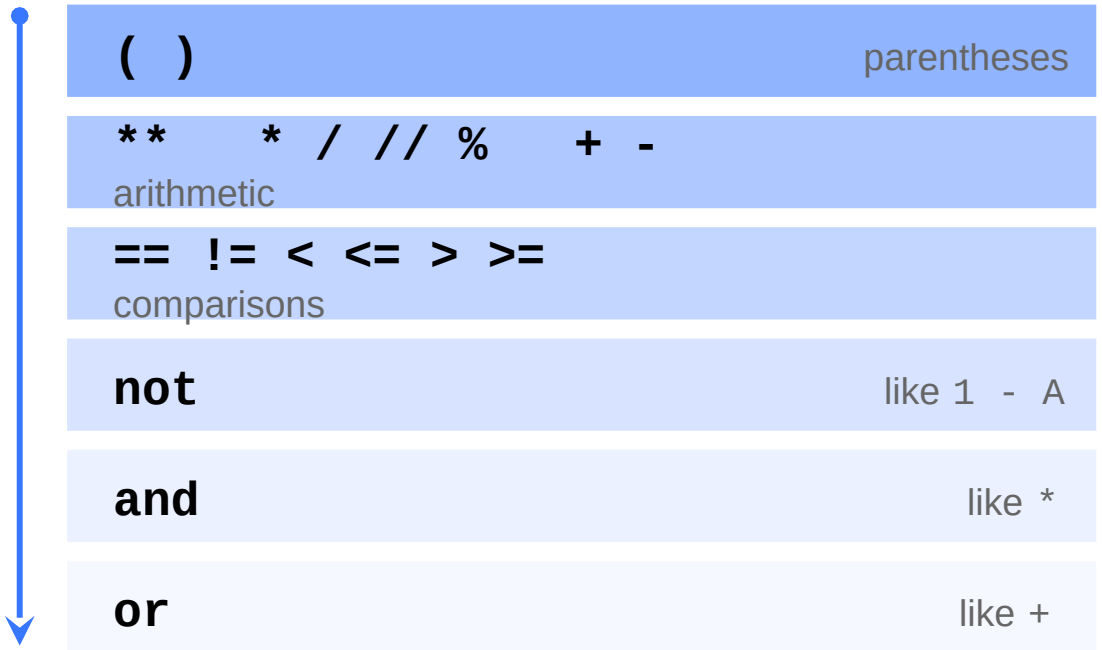
A	`not` A
0	1
1	0

`works like 1 - A`

Who binds first

Precedence groups the expression — before anything runs

binds first



binds last

one expression:

```
not 7 > 3 or 2 + 2 == 4
```

precedence adds the brackets, do it on paper to check:

```
(not (7 > 3)) or ((2 + 2) == 4)
```

nothing has run yet — *the grouping is decided first*

From left to right: compute what is required

Once the answer is known, Python stops — the rest is never evaluated

```
print(2 + 3 > 4 or 2 * 3 == 5 and 10 / 5 == 3)
```

```
>>> True
```

```
2 + 3 > 4 or 2 * 3 == 5 and 10 / 5 == 3
  5   > 4 or 2 * 3 == 5 and 10 / 5 == 3
    True  or 2 * 3 == 5 and 10 / 5 == 3
       True
```

The same expression, one sign flipped

False decides nothing for or — the right side must now be read

```
print(2 + 3 < 4 or 2 * 3 == 5 and 10 / 5 == 3)
```

```
>>> False
2 + 3 < 4 or 2 * 3 == 5 and 10 / 5 == 3
  5   < 4 or 2 * 3 == 5 and 10 / 5 == 3
    False  or 2 * 3 == 5 and 10 / 5 == 3
    False  or     6 == 5 and 10 / 5 == 3
    False  or       False and 10 / 5 == 3
    False  or       False
    False
```

The division that never happens

False and anything has already answered — 10 / 0 is never reached and the error is not raised

```
print(2 + 3 < 4 or 2 * 3 == 5 and 10 / 0 == 3)
```

```
>>> False
2 + 3 < 4 or 2 * 3 == 5 and 10 / 0 == 3
  5   < 4 or 2 * 3 == 5 and 10 / 0 == 3
    False  or 2 * 3 == 5 and 10 / 0 == 3
    False  or     6 == 5 and 10 / 0 == 3
    False  or       False and … (anything)
    False  or       False
    False
```

Python stops reading early

It answers as soon as it can — but the left side is always read

```
print(False and 10 / 0)
print(True or 10 / 0)
```

```
>>> False
>>> True
```

the danger on the right — never reached

```
print(10 / 0 and False)
print(10 / 0 or True)
```

```
ZeroDivisionError: division by zero
```

the danger on the left — always read first

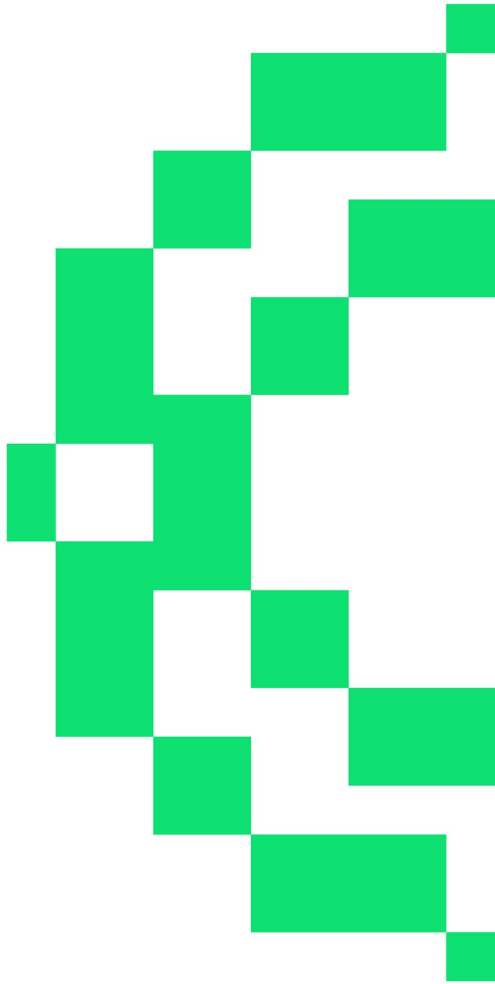
What does this print?

Commit before the run

```
print(not 7 > 3 or 2 + 2 == 4)
```

```
>>> True
```

```
"""A string can be sliced. Nothing goes back
in."""
```



02.

What lives inside a string

str is the odd type, remember?

LIVE

The first string behaviour: as a sequence of letters

Every letter is an item of the sequence, and has an index.

```
s = "computation"
print(s[0])
print(s[3])
```

```
>>> c
>>> p
```

Take care: in Python every first element has index ***zero***.

LIVE

Counting from the end

It counts the index backwards, from the last to the first position

```
s = "computation"
print(s[-1])
print(s[10])
print(s[-2])
print(s[11])
```

```
>>> n
>>> n
>>> o
IndexError: string index out of range
```

LIVE

Asking how long

The last index is len() minus one

```
s = "computation"
print(len(s))
print(s[len(s) - 1])
```

```
>>> 11
>>> n
```

Cutting a piece out

Start included, stop excluded

```
s = "computation is great"
print(s[0:4])
print(s[15:20])
```

```
>>> comp
>>> great
```

The lazy slice

Omitted means from the edge

```
s = "computation is great"
print(s[:4])
print(s[15:])
```

```
>>> comp
>>> great
```

Binding strings together — concatenation in Python

You have run this before, now you know what it is

```
print("comp" + "utation")
print("2" + "2")
```

```
>>> computation
>>> 22
```

```
print(7 * 2)
print("7" * 2)
print("comp" * 2)
```

```
>>> 14
>>> 77
>>> compcomp
```

LIVE

Is this inside a string?

A question with a one-word operator

```
s = "computation is great"
print("great" in s)
print("camp" in s)
```

```
>>> True
>>> False
```

LIVE

Python is case sensitive

Equal means identical

```
print("great" == "great")
print("great" == "Great")
```

```
>>> True
>>> False
```

```
s = "computation is great"
print("great" in s)
print("Great" in s)
```

```
>>> True
>>> False
```

LIVE

How strings compare

Letter by letter, left to right

```
print("apple" < "banana")
print("computation" < "computer")
```

```
>>> True
>>> True
```

LIVE

Ten before nine, revisited

The quotes were the whole crime scene — now you know why

```
print("10" < "9")
```

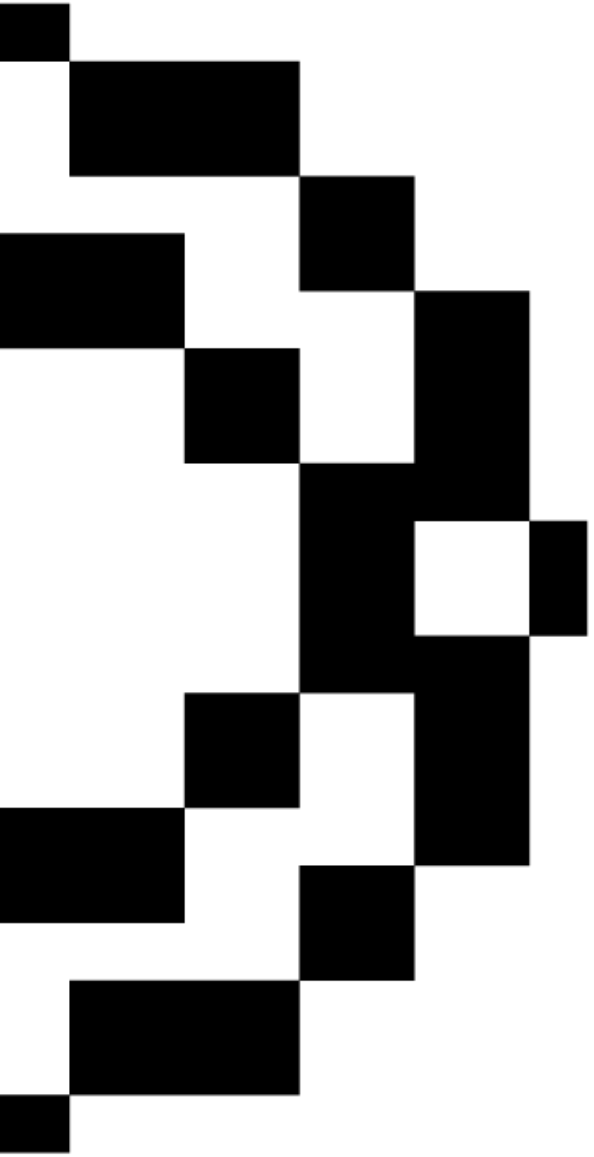
```
>>> True
```

Predict what this Python program outputs

Would you like to have this evaluate your final result in this course?

```
grade = "15"
print("Passed:", grade > "9")
```

```
>>> Passed: False
```



03.

Can a string be modified?

Pieces come out. Nothing goes in.

Try to modify one letter

That is when the sequence behaves like a unit — the second string face in Python

```
s = "computation"
s[0] = "C"
```

```
TypeError: 'str' object does not support item assignment
```

String methods give you a modified copy — never the original

You have used the dot before, with the math module. Here it reaches into a value

```
s.capitalize()
```

`s`	the object — the string, in our case
`.`	the dot — reach inside the object
`capitalize`	the method — it lives in the string's world
`()`	the call — it works on what is before the dot; it can be empty, or not, depending on what the method requires

LIVE

A method belongs to its value

You ask the string; it answers with a new one — the string itself does not change

```
s = "lisbon"
print(s.capitalize())
print(s)
```

```
>>> Lisbon
>>> lisbon
```

```
s = "lisbon"
s = s.capitalize()
print(s)
```

```
>>> Lisbon
```

Taking the spaces off the ends

strip is told nothing, and answers with a new string

```
city = "  lisbon  "
print( "." + city + ".")
print("." + city.strip() + ".")
print(len(city), len(city.strip()))
```

```
>>> .  lisbon  .
>>> .lisbon.
>>> 10 6
```

Only digits?

A question to ask before converting — it checks whether every character in the string is a digit

```
print("2026".isdigit())
print("20.26".isdigit())
print(int("2026") + 1)
```

```
>>> True
>>> False
>>> 2027
```

LIVE

Some methods need to be told more

What goes inside the parentheses: replace needs to know what to remove and what will replace that

```
s = "lizbon"
print(s.replace("z", "s"))
```

```
>>> lisbon
```

Python list — the other sequence that you have already seen

Same indexing, straight away

```
L = ["l", "i", "s", "b", "o", "n"]
print(L[0])
print(L[-1])
```

```
>>> l
>>> n
```

This one lets you modify its elements ...

The line strings refuse

```
L = ["l", "i", "s", "b", "o", "n"]
L[0] = "L"
print(L)
```

```
>>> ['L', 'i', 's', 'b', 'o', 'n']
```

… and can contain multiple types

A string holds only characters — a list holds anything

```
L = [31, "march", True]
L[0] = 5
print(L)
```

```
>>> [5, 'march', True]
```

Three questions, answered

In one sentence each

A question

combines with and, or, not — arithmetic first, then comparison, then logic operators

A string

is a sequence: every letter has an index, and the pieces come out with [ ]

A modification

is refused by strings — build a whole new string

Ken Thompson

1992

Every s[0] you wrote today came back as a letter. In memory, it was a number the whole time.

The placemat from 1992 decided how those numbers are stored — and no text written before it had to be rewritten.

Before the next theoretical lecture

On your own

Think Python — Sections 1.4 Strings · 5.2 Boolean Expressions · 5.3 Logical operators · 5.12 Debugging · 8.1 A string is a sequence · 8.2 String slices · 8.3 Strings are immutable · 8.4 String comparison · 8.5 String methods.

One habit — when a string surprises you, print `len()` and the index you thought you used.

Exercise 1 · Predict the value

Read the two lines below — and, or, not and the comparisons are this lecture's material. On paper, before any run: write what each line prints, and whether the program reaches its end. Then run it and compare.

```
print(7 > 3 and not 2 > 5)
print(1 + 1 == 2 or 10 / 0 > 1)
```

Exercise 2 · Predict what it prints

Read the program below — indexing is this lecture's material. One of these lines refuses to run. On paper: write what the first print shows, then say which line stops the program, and name the error. Then run it and compare.

```
s = "information"
print(s[0], s[-1], s[4])
print(s[11])
```

Exercise 3 · Extract the middle character

Write the line that prints the middle character of the string given in the panel. Use this lecture's indexing, together with len — the position is yours to compute. **One line**.

```
s = "computation"
# your line here
```

Exercise 4 · Clean it up

1) Write the line that prints LISBON — no spaces, all capitals — from the string given in the panel. Use this lecture's string methods; remember that each one answers with a new string. **One line does it**.

2) Write the line that prints Lisbon — no spaces, only the first capital letter. Again, **one line does it**.

```
city = "  lisbon  "
# your line here
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026

Exercise 5 · Predict, then run ★

Read the program below. On paper, before any run: write exactly what it prints. Then run it and compare.

```
date = "21/09/2026"

print(date.split("/"))
```

Exercise 6 · Run the back cover ★

Type the line every deck ends with, and run it. Watch what the program does — before your answer, and after. Then say, in one sentence, what the input() function did — without asking an AI.

```
input("Questions? ")
```